

Python函数进阶



目录

Contents

- ◆ 函数返回多个返回值
- ◆ 函数的多种参数
- ◆ 拆包
- ◆ 引用
- ◆ 匿名函数

学习目标

Learning Objectives

1. 知道函数如何返回多个返回值



函数返回多个返回值



思考

问：如果一个函数如些两个return（如下所示），程序如何执行

```
def return_num():  
    return 1  
    return 2  
  
result = return_num()  
print(result) # 1
```

答：只执行了第一个return，原因是因为return可以退出当前函数，导致return下方的代码不执行

多个返回值

如果一个函数要有多个返回值，该如何书写代码？

```
def return_num():  
    return 1, 2  
  
result = return_num()  
print(result)  # (1, 2)
```

- 注意：
 - ① return a, b写法，返回多个数据的时候，默认是元组类型
 - ② return后面可以连接列表、元组或字典，以返回多个值



目录

Contents

- ◆ 函数返回多个返回值
- ◆ 函数的多种参数
- ◆ 拆包
- ◆ 引用
- ◆ 匿名函数

学习目标

Learning Objectives

1. 掌握位置参数
2. 掌握关键字参数
3. 掌握不定长参数
4. 掌握缺省参数



函数的多种参数

函数参数种类

使用方式上的不同，函数有4中常见参数：

- 掌握位置参数
- 掌握关键字参数
- 掌握不定长参数
- 掌握缺省参数

位置参数

位置参数：调用函数时根据函数定义的参数位置来传递参数

```
def user_info(name, age, gender):  
    print(f'您的名字是{name}, 年龄是{age}, 性别是{gender}')
```



```
user_info('TOM', 20, '男')
```

注意：

传递的参数和定义的参数的顺序及个数必须一致

关键字参数

关键字参数：函数调用时通过“键=值”形式传递参数.

作用：可以让函数更加清晰、容易使用，同时也清除了参数的顺序需求.

```
def user_info(name, age, gender):  
    print(f'您的名字是{name}, 年龄是{age}, 性别是{gender}')
```



```
user_info('Rose', age=20, gender='女')  
user_info('小明', gender='男', age=16)
```

注意：

函数调用时，如果有位置参数时，位置参数必须在关键字参数的前面，但关键字参数之间不存在先后顺序

缺省参数

缺省参数：缺省参数也叫默认参数，用于定义函数，为参数提供默认值，调用函数时可不传该默认参数的值（注意：所有位置参数必须出现在默认参数前，包括函数定义和调用）。

作用：当调用函数时没有传递参数，就会使用默认是用缺省参数对应的值。

```
def user_info(name, age, gender='男'):  
    print(f'您的名字是{name}, 年龄是{age}, 性别是{gender}')
```



```
user_info('TOM', 20)  
user_info('Rose', 18, '女')
```

注意：

函数调用时，如果为缺省参数传值则修改默认参数值，否则使用这个默认值

不定长参数

不定长参数：不定长参数也叫可变参数。用于不确定调用的时候会传递多少个参数(不传参也可以)的场景。

作用：当调用函数时不确定参数个数时，可以使用不定长参数

不定长参数的类型：

①位置传递

②关键字传递

位置传递

```
def user_info(*args):  
    print(args)  
  
# ('TOM',)  
user_info('TOM')  
# ('TOM', 18)  
user_info('TOM', 18)
```

注意：

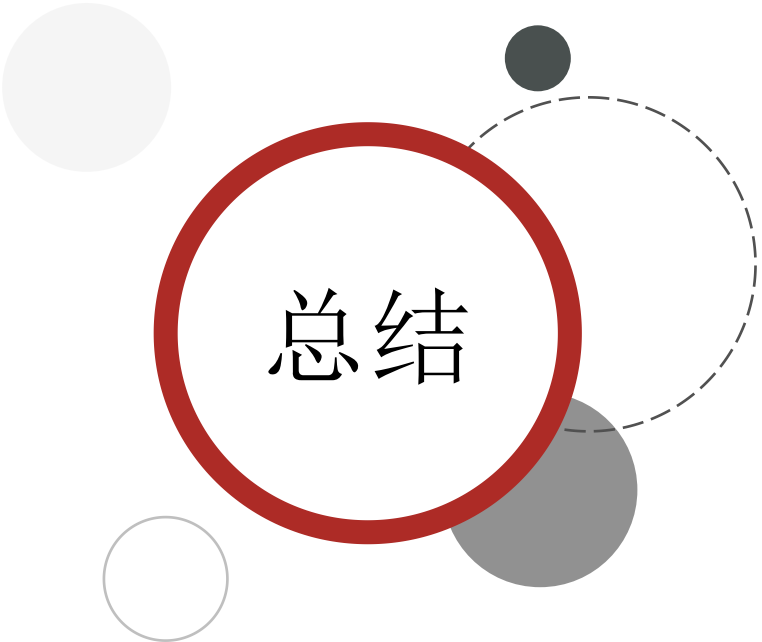
传进的**所有参数**都会被**args变量收集**，它会根据传进参数的位置合并为一个元组(tuple)，**args是元组类型**，这就是**位置传递**

关键字传递

```
def user_info(**kwargs):  
    print(kwargs)  
  
# {'name': 'TOM', 'age': 18, 'id': 110}  
user_info(name='TOM', age=18, id=110)
```

注意：

参数是“键=值”形式的形式的情况下，所有的“键=值”都会被kwargs接受，同时会根据“键=值”组成字典。



总结

1. **掌握位置参数**
 - 根据参数位置来传递参数
2. **掌握关键字参数**
 - 通过“键=值”形式传递参数
3. **掌握缺省参数**
 - 不传递参数值时会使用默认的参数值
4. **掌握不定长参数**
 - 位置传递 args 元组的形式接受参数
 - 关键字传递 kwargs 字典的形式接受参数



目录

Contents

- ◆ 函数返回多个返回值
- ◆ 函数的多种参数
- ◆ 拆包
- ◆ 引用
- ◆ 匿名函数

学习目标

Learning Objectives

1. 掌握拆包的用法
2. 掌握交换变量值的方法
3. 知道可变和不可变类型有哪些



拆包

拆包

拆包：对于函数中的多个返回数据，去掉元组，列表 或者字典 直接获取里面数据的过程

对元组拆包：

```
my_tuple = (1, 3.14, "hello", True)
num, pi, my_str, my_bool = my_tuple
```

一个元组中有多个元素，当我们想要获取元组中的每一个元素的时候就可以使用拆包获取

拆包

对函数的多个函数值拆包：当函数有多个返回值的时候，返回值会组成一个元组

```
def return_num():  
    return 100, 200  
  
num1, num2 = return_num()  
  
print(num1) # 100  
print(num2) # 200
```

对列表拆包：

```
my_list = [1, 3.14, "hello", True]  
num, pi, my_str, my_bool = my_list
```

拆包

对字典拆包:

```
my_dict = {"name": "老王", "age": 19}
ret1, ret2 = my_dict
```

对字典拆包的时候，这里的ret1，ret2获取的是字典的key值。也就是 'name' 和 'age'



变量数值交换

变量数值交换

拆包应用除了获取各种容器中的元素外，还可以使用在变量的数值交换上

需求：有变量a = 10和b = 20，交换两个变量的值

- 方法一
- 借助第三变量存储数据

```
# 1. 定义中间变量
c = 0
# 2. 将a的数据存储到c
c = a
# 3. 将b的数据20赋值到a，此时a = 20
a = b
# 4. 将之前c的数据10赋值到b，此时b = 10

b = c print(a) # 20
print(b) # 10
```

变量数值交换

需求：有变量 $a = 10$ 和 $b = 20$ ，交换两个变量的值

- 方法二
- 拆包

```
a, b = 1, 2  
  
a, b = b, a  
  
print(a) # 2  
print(b) # 1
```



目录

Contents

- ◆ 函数返回多个返回值
- ◆ 函数的多种参数
- ◆ 拆包
- ◆ 引用
- ◆ 匿名函数



学习目标

Learning Objectives

1. 了解什么是引用



引用

什么是引用

在我们的Python程序中任何的变量的使用都是要开销电脑的内存资源的

问题：

```
a = 10
```

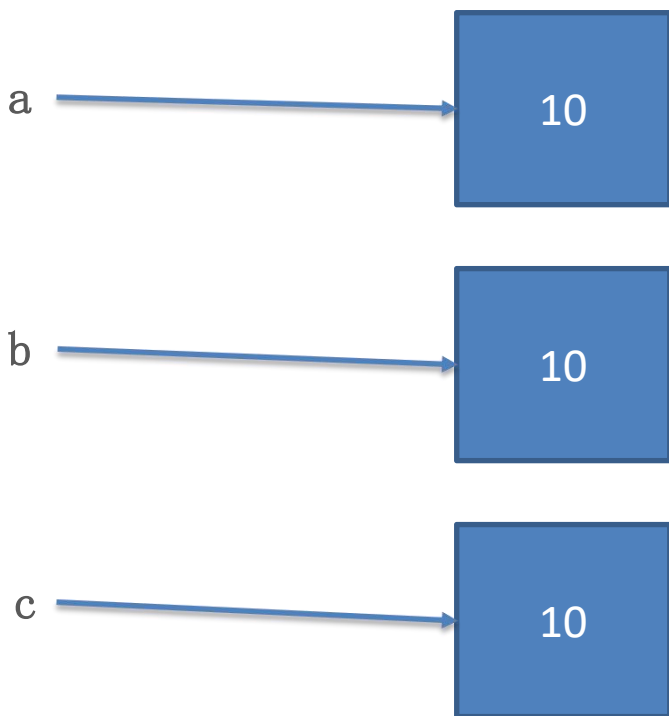
```
b = a
```

```
c = a
```

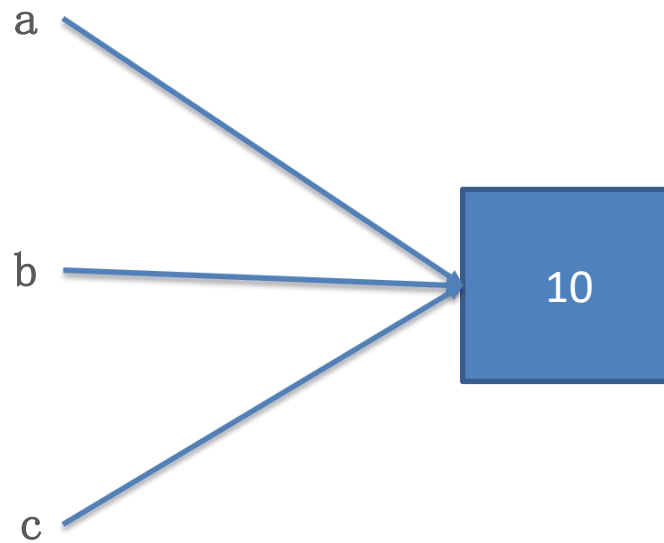
a , b, c这三个变量对应的数值都是10那么他们会使用多少的内存资源呢？

什么是引用

两种猜想:



猜想1: a, b, c三个变量分别开销三块内存资源



猜想2: a, b, c三个变量共用一块内存资源

引用传递

为了验证猜想，我们可以用`id()`来判断变量是否为同一个地址，这里说的地址就可以理解为引用

```
a = 10
```

```
b = a
```

```
c = a
```

```
id(a)
```

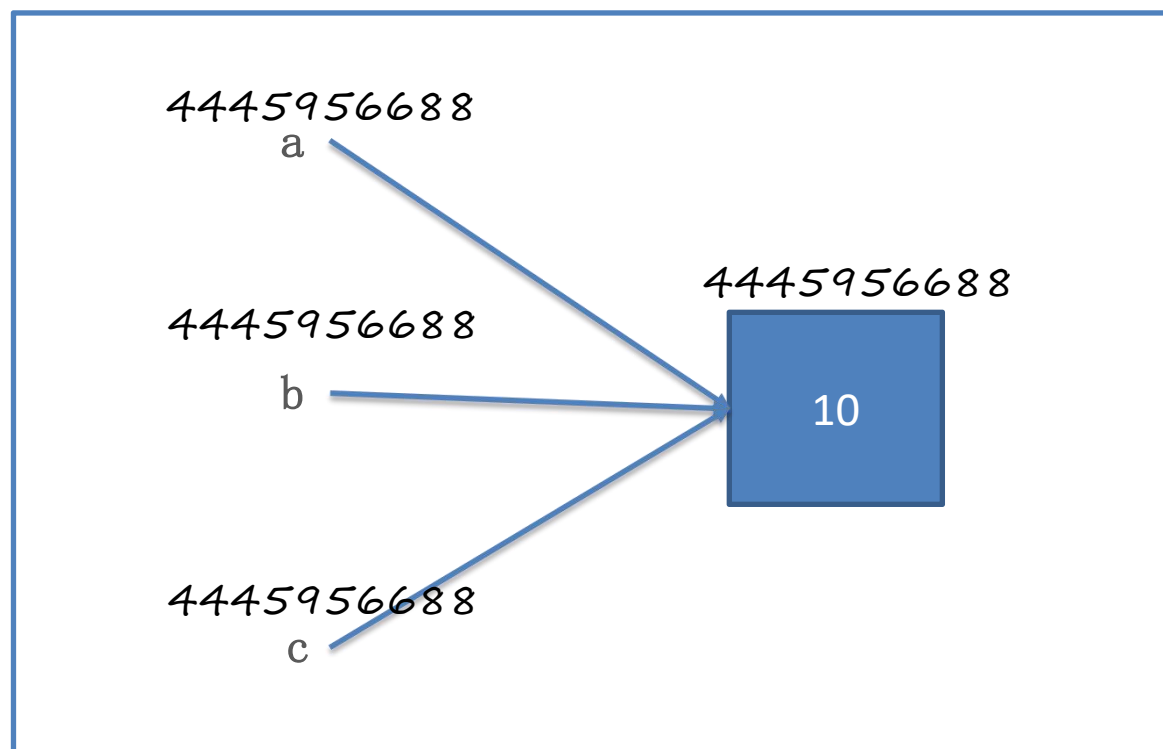
```
➔4445956688
```

```
id(b)
```

```
➔4445956688
```

```
id(c)
```

```
➔4445956688
```



这里可以看到`a`, `b`, `c`的地址是一样的，也就是说它们共用一个地址(引用)

结论：Python中在使用 “=” 进行赋值传递的时候传递的是地址，也就是引用传递

函数参数的引用传递

```
def func(data):  
    # 这里的data接受的是num的地址，所以num和data的地址一样  
    print("data:", id(data))  
  
# 创建一个变量  
num = 10  
# 打印num的地址  
print("num:", id(num))  
  
# 调用函数func  
func(num)
```

4380703680

10

执行结果:

```
num: 4380703680  
data: 4380703680
```

可变类型与不可变类型

所谓可变类型与不可变类型是指：数据能够直接进行修改，如果能直接修改那么就是可变，否则是不可变

➤ 可变类型

- ❑ 列表
- ❑ 字典
- ❑ 集合

➤ 不可变类型

- ❑ 整型
- ❑ 浮点型
- ❑ 字符串
- ❑ 元组



传智教育旗下高端IT教育品牌